

```

; multitasking and dynamic thread registration
[org 0x0100]
jmp start
; PCB layout:
; ax,bx,cx,dx,si,di,bp,sp,ip,cs,ds,ss,es,flags,next,dummy
; 0, 2, 4, 6, 8,10,12,14,16,18,20,22,24, 26 , 28 , 30
pcb: times 32*16 dw 0 ; space for 32 PCBs
stack: times 32*256 dw 0 ; space for 32 512 byte stacks
nextpcb: dw 1 ; index of next free pcb
current: dw 0 ; index of current pcb
lineno: dw 0 ; line number for next thread
,,,,, COPY LINES 028-071 FROM EXAMPLE 10.1 (printnum) ,,,,,
n: dw 5
counter: dw 0

```

```

clrscr: push es
push ax
push di

```

```

mov ax, 0xb800
mov es, ax ; point es to video base
mov di, 0 ; point di to top left column
nextloc: mov word [es:di], 0x0720 ; clear next char on screen
add di, 2 ; move to next screen location
cmp di, 4000 ; has the whole screen cleared
jne nextloc ; if no clear next position
pop di
pop ax
pop es
ret

```

```

printnum: push bp
mov bp, sp
push es
push ax
push bx
push cx
push dx
push di
mov di, 80 ; load di with columns per row
mov ax, [bp+8] ; load ax with row number
mul di ; multiply with columns per row

```

```

mov di, ax ; save result in di
add di, [bp+6] ; add column number
shl di, 1 ; turn into byte count
add di, 8 ; to end of number location
mov ax, 0xb800
mov es, ax ; point es to video base
mov ax, [bp+4] ; load number in ax
mov bx, 16 ; use base 16 for division
mov cx, 4 ; initialize count of digits
nextdigit: mov dx, 0 ; zero upper half of dividend
div bx ; divide by 10
add dl, 0x30 ; convert digit into ascii value
cmp dl, 0x39 ; is the digit an alphabet
jbe skipalpha ; no, skip addition
add dl, 7 ; yes, make in alphabet code
skipalpha: mov dh, 0x07 ; attach normal attribute
mov [es:di], dx ; print char on screen
sub di, 2 ; to previous screen location
loop nextdigit ; if no divide it again
pop di
pop dx
pop cx
pop bx
pop ax
pop es
pop bp
ret 6

```

```

; mytask subroutine to be run as a thread
; takes line number as parameter
mytask: push bp
mov bp, sp
sub sp, 2 ; thread local variable
push ax
push bx
mov ax, [bp+4] ; load line number parameter
mov bx, 0 ; use column number 0
mov word [bp-2], 0 ; initialize local variable
printagain: push ax ; line number
push bx ; column number
push word [bp-2] ; number to be printed
call printnum ; print the number
inc word [bp-2] ; increment the local variable
jmp printagain ; infinitely print

```

```

pop bx
pop ax
mov sp, bp
pop bp
ret
; subroutine to register a new thread
; takes the segment, offset, of the thread routine and a parameter
; for the target thread subroutine
initpcb: push bp
mov bp, sp
push ax
push bx
push cx
push si
mov bx, [nextpcb] ; read next available pcb index
cmp bx, 32 ; are all PCBs used
je exit ; yes, exit

mov cl, 5
shl bx, cl ; multiply by 32 for pcb start
mov ax, [bp+8] ; read segment parameter
mov [pcb+bx+18], ax ; save in pcb space for cs
mov ax, [bp+6] ; read offset parameter
mov [pcb+bx+16], ax ; save in pcb space for ip
mov [pcb+bx+22], ds ; set stack to our segment
mov si, [nextpcb] ; read this pcb index
mov cl, 9
shl si, cl ; multiply by 512
add si, 256*2+stack ; end of stack for this thread
mov ax, [bp+4] ; read parameter for subroutine
sub si, 2 ; decrement thread stack pointer
mov [si], ax ; pushing param on thread stack
sub si, 2 ; space for return address
mov [pcb+bx+14], si ; save si in pcb space for sp
mov word [pcb+bx+26], 0x0200 ; initialize thread flags
mov ax, [pcb+28] ; read next of 0th thread in ax
mov [pcb+bx+28], ax ; set as next of new thread
mov ax, [nextpcb] ; read new thread index
mov [pcb+28], ax ; set as next of 0th thread
inc word [nextpcb] ; this pcb is now usedq
exit: pop si
pop cx
pop bx
pop ax

```

```
pop bp
ret 6
; timer interrupt service routine
timer:
push ds
push bx
push cs
pop ds
```

```
mov bx, [current]
shl bx, 5
mov [pcb+bx+0], ax
mov [pcb+bx+4], cx
mov [pcb+bx+6], dx
mov [pcb+bx+8], si
mov [pcb+bx+10], di
mov [pcb+bx+12], bp
mov [pcb+bx+24], es
pop ax
mov [pcb+bx+2], ax
pop ax
mov [pcb+bx+20], ax
pop ax
mov [pcb+bx+16], ax
pop ax
mov [pcb+bx+18], ax
pop ax
mov [pcb+bx+26], ax
mov [pcb+bx+22], ss
mov [pcb+bx+14], sp
```

```
inc word [counter] ; increment counter
mov ax, [counter]
cmp ax, [n] ; reached n
jl resume_current
```

```
mov word [counter], 0
```

```
mov bx, [pcb+bx+28]
mov [current], bx
shl bx, 5
```

```
resume_current:
```

```
mov cx, [pcb+bx+4]
mov dx, [pcb+bx+6]
mov si, [pcb+bx+8]
mov di, [pcb+bx+10]
mov bp, [pcb+bx+12]
mov es, [pcb+bx+24]
mov ss, [pcb+bx+22]
mov sp, [pcb+bx+14]
push word [pcb+bx+26]
push word [pcb+bx+18]
push word [pcb+bx+16]
push word [pcb+bx+20]
mov al, 0x20
out 0x20, al
mov ax, [pcb+bx+0]
mov bx, [pcb+bx+2]
pop ds
iret
```

start:

```
call clrscr
```

```
xor ax, ax
mov es, ax ; point es to IVT base
cli
mov word [es:8*4], timer
mov [es:8*4+2], cs ; hook timer interrupt
sti
nextkey: xor ah, ah ; service 0 – get keystroke
int 0x16 ; bios keyboard services
push cs ; use current code segment
mov ax, mytask
push ax ; use mytask as offset
push word [lineno] ; thread parameter
call initpcb ; register the thread
inc word [lineno] ; update line number
jmp nextkey ; wait for next keypress
```

5ED9
AB2B
F68C
42D7
BC23
620F
620E
70BE
8F47
8F47
8F47
8F47
8F48
8F48
8F47
620F
620F
620F
620F
620F
620F
620F
620F
620F
620F